Suggested computational setup

14.33 Research and Communication in Economics · Fall 2026

The requirements below apply to every project. The named tools and editor choices are suggestions.

Required for every project

- **Open source analysis software.** Consult the professor and TA and obtain clearance before using Stata or other proprietary analysis software.
- **Scripted results.** Script data processing, analysis, tables, and figures. Script collection wherever access permits and document unavoidable manual steps. Provide one command that rebuilds results from saved inputs in a fresh session.
- **Pinned versions.** Record exact language, package, and external-tool versions in a committed environment recipe or lockfile. Set seeds for random procedures.
- **Git tracking.** Commit and push code, environment files, and documentation. Keep secrets and large or restricted datasets outside Git.
- **Portable, documented inputs.** Use project-relative paths. Preserve raw data unchanged and record their source and version.

1. Choose a language and environment

I recommend **R or Python**. Choose the one you know better or whose packages fit your project. Use a project environment containing the language and packages it needs, and commit a lockfile recording their versions.

Suggested tool	Good fit	Files to commit	Recreate the environment
Pixi	R, Python, or a combination. Convenient when a project also needs other applications or command-line tools	`pixi.toml` and `pixi.lock`	`pixi install --locked`
uv	Python projects	`pyproject.toml`, `uv.lock`, and `.python-version`	`uv sync --locked`

With Pixi, initialize using `pixi init`. Add packages with `pixi add PACKAGE` or `pixi add r-PACKAGE` for R. Use `pixi add --pypi PACKAGE` for Python packages available only on PyPI. Use `pixi run --locked` to run programs in the project environment.
With uv, initialize using `uv init --bare`, select a Python version using `uv python pin VERSION`, and add packages with `uv add PACKAGE`. Use `uv run --locked` to run Python in the project environment. Install and record external tools separately.
Add dependencies through your chosen manager so the recipe and lockfile stay current. Review and commit both after changes, then rerun the project. If an essential R package is unavailable through Pixi, discuss renv with the course staff. An existing setup is also suitable if it records exact versions and supports reproducible runs.

2. Suggested editor and terminal setup

I suggest VS Code. Open the repository through **File → Open Folder**, then use **Terminal → New Terminal** for its integrated terminal. Keep your script, terminal, and output preview visible in one window. You can also run a coding agent in a separate terminal tab.

Two ways to work with a script

Both workflows below use an ordinary **`.py` or `.R` script file**. Save your analysis code in that file.
Run the whole script and inspect saved outputs. From the repository root, run `uv run --locked python code/SCRIPT.py`, `pixi run --locked python code/SCRIPT.py`, or `pixi run --locked Rscript code/SCRIPT.R`, depending on your setup. Replace `SCRIPT.py` or `SCRIPT.R` with your script's filename. The terminal shows messages and errors. Have the script save tables and figures to the configured output directory. Open a saved PDF beside the script using a PDF viewer extension, or open a PNG or CSV directly in VS Code. Save edits, rerun the command, and refresh the preview if needed. Figures need an explicit save command, such as `fig.savefig(...)` in Python or `ggsave(...)` in R.
Run selected lines in an interactive interpreter. An interpreter is a Python or R session that retains objects between commands. From the repository root, start one in an integrated terminal using the project environment and the configuration described in section 3:

Language	Start the session
Python	`uv run --locked python -i code/config.py` or `pixi run --locked python -i code/config.py`
R	`pixi run --locked R --no-save --no-restore`

Python's `-i` runs the configuration file and leaves the interpreter open. In R, send the `source("code/config.R")` line first.
With that interpreter terminal active, highlight a complete statement or block in your script. Open the Command Palette with **Cmd+Shift+P** on macOS or **Ctrl+Shift+P** on Windows/Linux, then choose **Terminal: Run Selected Text in Active Terminal**. Inspect objects and printed results in the interpreter. Plots appear in a graphics window or a configured plot viewer. Run the script's imports first. You are working from a script, without creating a Jupyter `.ipynb` file.
For Python extension features, use **Python: Select Interpreter** to choose `.venv/bin/python` with uv or `.pixi/envs/default/bin/python` with Pixi. On Windows, choose `.venv/Scripts/python.exe` or `.pixi/envs/default/python.exe`. After interactive exploration, save the useful code and run the entire script in a fresh session to check that it works from start to finish.
For a notebook-like workflow in **Python**, consider marimo, which saves notebooks as `.py` files. See the notebooks memo. It does not support R.

Debugging and Git history

- **Debug a script:** With Python Debugger, click beside a line number to set a breakpoint, where execution pauses. Use **Run → Start Debugging** and choose **Python File**. Inspect **Variables**, add expressions to **Watch**, and use **Step Over** or **Step Into** to follow execution. Use the project interpreter. The optional R Debugger extension provides similar tools for R and needs the `vscDebugger` R package in the project environment.
- **Inspect changes and the Git tree:** Open **Source Control** and click a changed file to see a side-by-side diff. Its Source Control Graph shows commits and branch relationships. Select a commit, then a file, to inspect that change. For one file's history, open **Explorer → Timeline**. These built-in views help you review an agent's edits and trace when a result changed.

Terminal commands

The shell runs commands that start programs or manage files. A Python or R prompt accepts that language's code. Use a separate shell terminal for the commands below while an interpreter is running.

Task	macOS/Linux, with zsh or bash	Windows, with PowerShell
Show the current folder	`pwd`	`Get-Location`
List files and folders	`ls`	`Get-ChildItem`
Enter the code folder	`cd code`	`Set-Location code`
Move up one folder	`cd ..`	`Set-Location ..`
Create folders, including missing parents	`mkdir -p data/raw`	`New-Item -ItemType Directory -Force data/raw`
Show Git status	`git status`	`git status`
Show unstaged changes	`git diff`	`git diff`
Stop a running command	**Ctrl+C**	**Ctrl+C**

The Pixi and uv commands above work on both platforms. Quote paths containing spaces. Use **Tab** to complete names and ↑ to recall commands. Ask your coding agent to explain unfamiliar commands and flags.
Optional terminal conveniences:

- **Terminal apps:** iTerm2 on macOS or Ghostty on macOS/Linux.
- **Suggestions and highlighting:** For zsh, Oh My Zsh manages shell configuration. Add zsh-autosuggestions and zsh-syntax-highlighting for those features.
- **Persistent remote sessions:** tmux keeps terminal sessions available when you disconnect and reconnect.

3. Organize the project

Create your repository from the course template using the GitHub memo. Extend its folders as needed. A suggested layout is:

```
project/
├── README.md          Setup, data access, run command, and expected outputs
├── .gitignore         Data, secrets, environments, and caches to exclude
├── code/              Scripts and one shared configuration file
├── data/raw/          Unchanged source files
├── data/clean/        Generated analysis data
├── docs/              Source notes, codebook, and important decisions
└── output/            Generated tables, figures, and checks
```

Keep environment files at the root. The README should let someone recreate the environment, obtain the exact inputs, and run the project. Save variable definitions, units, missing-value codes, and observation keys in a codebook.

Project-relative paths

Keep paths and shared settings in **one configuration file for your chosen language**. Each script imports or sources it so root-finding logic lives in one place. Define shared directories there. Document that full-script commands run from the repository root. Avoid machine-specific paths and `setwd()`.

Python: `code/config.py` uses pathlib to locate the root from the configuration file's own location.

```python
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
RAW = ROOT / "data" / "raw"
CLEAN = ROOT / "data" / "clean"
OUTPUT = ROOT / "output"
```

At the top of scripts saved directly in `code/`, import the settings you need:

```python
from config import RAW, CLEAN, OUTPUT
```

This import works from a script in `code/` or an interpreter started with `python -i code/config.py` from the root.

R: `code/config.R` uses here to find the project root automatically, using the Git repository as a marker. Add `here` to the project environment.

```r
RAW <- here::here("data", "raw")
CLEAN <- here::here("data", "clean")
OUTPUT <- here::here("output")
```

At the top of each R script, load that configuration from the repository root:

```r
source("code/config.R")
```

Build file paths from these shared directories. Use `RAW` for source data, `CLEAN` for processed data, and `OUTPUT` for tables, figures, and checks. Document the inputs and outputs of each stage. If data must live elsewhere, read that location from an environment variable and document it. Keep credentials in environment variables too.

Local data and generated files

Exclude large datasets, credentials, installed environments, and session files from Git. A starting `.gitignore` is:

```
/data/raw/
/data/clean/
*.parquet
*.duckdb
.venv/
.pixi/
__pycache__/
.RData
.Rhistory
.env
.env.*
```

Record source URLs, retrieval dates, releases, queries, and filenames. Back up local data in a permitted storage location. Rebuild analysis from the saved snapshot so a rerun does not silently use newer data.

4. Suggested packages and extensions

The tools below are optional. Add them as your project needs them.

Data collection, storage, and checks

Task	Suggested tools	Use
Tabular cleaning and summaries	**R:** dplyr. **Python:** pandas	Select, filter, recode, group, and join tables
Efficient processing of larger tables	**R:** data.table. **Python:** Polars	Efficient joins and grouped calculations. Choose based on your data and workflow
Parquet files	**R:** arrow. **Python:** PyArrow	Store large tables in a compressed column-based format. Export small CSV extracts for inspection
SQL and data larger than RAM	**R and Python:** DuckDB	Query Parquet, CSV, and JSON directly. Filter, summarize, and join before loading results. Can use disk for larger-than-memory work
Spatial data	**R:** sf. **Python:** GeoPandas	Read spatial files, calculate distances, and join locations to areas. Align coordinate reference systems first
Additional geospatial tools	GDAL and PyQGIS	Raster and vector conversion with GDAL, or automation of QGIS operations with PyQGIS
APIs and downloads	**R:** httr2. **Python:** HTTPX	Script requests, authentication, pagination, error handling, and retries. Respect rate limits
Static websites	**R:** rvest. **Python:** Beautiful Soup	Extract fields from saved HTML. Look for bulk downloads or an API first
JavaScript-rendered websites	**Python:** Playwright and Crawlee	Playwright automates a browser. Crawlee helps organize larger crawls. Use when simpler requests do not retrieve the content
PDFs with a text layer	**R:** pdftools. **Python:** PyMuPDF or pdfplumber	Extract existing text, coordinates, and layout information before trying OCR
Tables in text-based PDFs	**R:** tabulapdf. **Python:** Camelot or pdfplumber	Extract tabular content. Check parsed rows and columns against the original pages
Scanned or complex documents	Docling with an OCR engine such as Tesseract	Recover text and document structure from scans. Validate a random sample and difficult layouts by hand
Searchable scanned PDFs	OCRmyPDF	Add a searchable text layer. Further extraction is needed to produce structured tables

Task	Suggested tools	Use
Model-coded text	**Python:** Expected Parrot's EDSL	Script model calls and save prompts, model settings, and responses. Pilot accuracy and cost, then validate against human coding
Data validation	**R:** pointblank. **Python:** pandera	Check schemas, keys, types, allowed values, missingness, and ranges. Also check merge relationships and report unmatched rows
Function tests	**R:** testthat. **Python:** pytest	Test consequential logic with small examples whose answers you know

If selective reads and efficient processing still exceed your laptop's capacity, consider MIT's Engaging cluster.

Statistical analysis and output

Task	Suggested tools	Use
Regression and fixed effects	**R:** fixest. **Python:** PyFixest or statsmodels	fixest and PyFixest for applied econometric models with fixed effects. statsmodels for a broader range of statistical models
Prediction and machine learning	**R:** tidymodels. **Python:** scikit-learn	Preprocessing, model fitting, and out-of-sample evaluation
Time series	**R:** fable. **Python:** tsecon or statsmodels	fable for forecasting workflows. tsecon for time-series econometrics. statsmodels for standard time-series models
Difference-in-differences	**R:** did or fixest. **Python:** diff-diff or PyFixest	Choose the estimator for your treatment timing and assumptions, including staggered adoption
Marginal effects and predictions	**R and Python:** marginaleffects	Predictions, comparisons, and slopes after estimation
Regression discontinuity	**R and Python:** RD Packages	Specialized estimation and diagnostics, including `rdrobust` and `rddensity`
Double/debiased machine learning	**R and Python:** DoubleML	Estimate supported causal models with machine learning for nuisance functions
Figures	**R:** ggplot2. **Python:** matplotlib and seaborn	Script plots with clear labels, units, and sample definitions
Binscatters	**R and Python:** binsreg	Binned scatterplots with statistical inference
Tables	**R:** modelsummary and gt. **Python:** MakeTables and Great Tables	modelsummary and MakeTables organize model results. gt and Great Tables format tables for export
Computational documents	**R and Python:** Quarto	Render code and saved outputs into reports. Follow the course's separate rules for drafting paper prose
Pipeline automation	GNU Make	Record dependencies between inputs and output files and rerun affected steps. A simple scripted entry point is also suitable

VS Code extensions

Install the language support you use, then add conveniences as needed. VS Code already includes Git integration and a basic Markdown preview.

Extension	Useful for
Python and Pylance	Python execution, environment selection, completion, and diagnostics
Python Debugger	Breakpoints, stepping through scripts, variable inspection, and watch expressions. Installed with the Python extension
R	Send selected code to R and inspect objects. Add `languageserver` for completion and `httpgd` for the plot viewer
R Debugger	Optional breakpoints and step controls for R scripts. Requires the `vscDebugger` package and configuration for your project's R executable
Ruff	Python formatting and linting, which flags common code problems
Rainbow CSV	Inspect columns in small CSV or TSV files
PDF viewer	Open saved PDF figures and reports beside the script that creates them
Quarto	Edit and preview `.qmd` files. Requires the Quarto command-line tool
LaTeX Workshop	Edit, compile, and preview LaTeX documents when using a TeX installation
AIChatRender	Read Claude Code or Codex CLI transcripts with rendered Markdown and math. Automatic terminal following is platform-dependent
autoDocstring	Create Python function documentation scaffolds. Fill in and check their descriptions
Markdown Preview Enhanced	Preview Markdown with math, diagrams, and additional export options
Markdown Preview Mermaid Support	Display Mermaid diagrams in VS Code's built-in Markdown preview